Sudoku Solver: Use Case Diagram and Description

EECS 348 Software Engineering I, Assignment 5

Author: Zach Lowe Revision date: 10/29/2026

1. Revised use case diagram

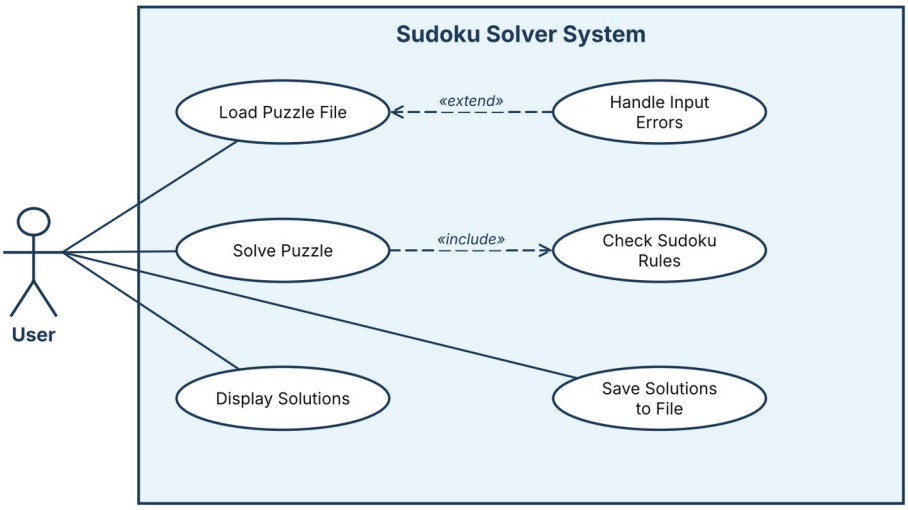


Figure 1. Solid lines are associations with the User. Dashed arrows are <<include>> and <<extend>> relationships.

2. Changes made to the original diagram

- Renamed the system boundary from "Grocery cart system" to "Sudoku Solver System".
- Changed the actor from "host" to "User", so the diagram matches the text.
- Replaced the included use case "check validity backtracking algorithm" with "Check Sudoku Rules". The backtracking itself stays in Solve Puzzle.
- Made "Handle errors" an <<extend>> of Load Puzzle File, since it only happens when a file cannot be loaded.
- Added "Save Solutions to File" for the Assignment 5 solutionN.txt output.
- Moved "no repeated numbers" from the precondition to an alternative flow, and treated "no solution" as a normal result rather than an error.

3. Use case description

(a) Use case identified

Solve Sudoku Puzzle. It is the main goal of the User, and the other use cases support it.

(b) Brief description

The User runs the solver. The program reads each puzzle file, finds every solution by recursive backtracking, shows the solutions on screen and saves them to solutionN.txt, or reports that there is no solution.

(c) Step-by-step flow of events

1. The User runs the program, which processes puzzle1.txt through puzzle5.txt one after another.
2. The program loads the puzzle file into a 9x9 board (Load Puzzle File).
3. The program checks that the given digits do not already repeat in a row, column or 3x3 grid.
4. The program finds the next blank cell and tries the digits 1 to 9, using Check Sudoku Rules for each one.
5. If a digit is allowed, the program places it and moves to the next blank cell. If no digit fits, it backs up and tries the next digit in the previous cell.
6. When every cell is filled, the program confirms the board is a valid solution, then displays it and saves it.
7. The search continues until every possibility has been tried, so all solutions are found.
8. The program reports how many solutions were found, or that there is no solution.
9. The program moves on to the next puzzle file.

(d) Detailed description

Use case	UC-1 Solve Sudoku Puzzle
Actor	User
Description	The User runs the solver on five puzzle files. For each file the program finds every solution by exhaustive recursive backtracking, shows the solutions on screen, and saves them to solutionN.txt. A puzzle can have more than one solution, so all of them are reported.
Trigger	The User runs the program.
Preconditions	The puzzle files puzzle1.txt to puzzle5.txt are in the folder the program is run from and can be read.
Postconditions	Solutions found: the screen shows the original puzzle and every solution, and solutionN.txt holds every solution followed by "Solutions found: n". No solution: the screen shows "No solution found." and solutionN.txt holds only that line.
Alternative flows	2a. The file is missing or badly formed (wrong number of rows or columns, or an invalid character): the program prints an error message, skips that puzzle and goes to the next one (Handle Input Errors). No solution file is written for it. 3a. The given digits already conflict: the search is skipped and the result is "No solution found." 6a. The solution file cannot be created: the program prints an error message and still shows the solutions on screen.
Relationships	Includes Check Sudoku Rules. Uses Load Puzzle File, Display Solutions and Save Solutions to File. Handle Input Errors extends Load Puzzle File.